

Fiches de réponse jury — Soutenance CloudShift

Document de préparation. Tous les chiffres ci-dessous ont été vérifiés dans le code (session de correction post-audit). Les valeurs ici sont celles du rapport corrigé.

Format de chaque fiche : Q (question probable) · Réponse · Preuve dans le code · À éviter.

0. CHIFFRES-CLÉS À MÉMORISER (vérifiés)

Élément	Valeur exacte	Preuve
Modèles LLM	gpt-5.1 (Agents 01, 02), gpt-5.4-mini (Agent 03)	03.py : AZUREMODEL0x
Services Docker	7 (postgres, neo4j, vault, vault-unsealer, apideck)	docker-compose.yml
Réseaux Docker	internalnet, externalnet	docker-compose.yml
Pipeline LangGraph	22 nœuds, 9 routeurs conditionnels	pipeline/pipeline_graph.py
Tests	661 cas collectés (pytest), 31 fichiers, 620 fonctions	pytest --collect-only
Couverture	35 % global (core/ 48 %)	pytest-cov (à relancer le jour J)
Outils par agent	Agent 01 = 8 · Agent 02 = 4 · Agent 03 = 6	listes_TOOLS
SAW-AHP	poids 0,4253 / 0,2618 / 0,1631 / 0,1032 / 0,0466	0466/ahp_weights.py
Embeddings	all-mpnet-base-v2, 768D, local	rag/rag_config.py
MAX_CORRECTIONS	5 (porté de 3→5)	core/constants.py
Classes d'erreur deploy	7 (STATE, QUOTA, ENVIRONMENTAL, DEPENDENCY, SEMANTIC, ...)	agents/deploy.py
SecurityPolicyEngine	13 types de ressources	agents/iacgenerator/securitypolicy_engine.py
Rate limiting	5/min (login), 10/min (autres endpoints sensibles)	services/routers/v1/auth_router.py
Invitation	token valable 72 h	auth_router.py (timedelta(hours=72))
Python	3.12 (Dockerfile)	Dockerfile
Provider azurerm	~> 3.99	annexe B

A. MODÈLE LLM (le piège n°1 — désamorcé par la correction)

Q : Quel modèle exactement, et pourquoi le rapport mentionnait GPT-4.1 ?

« Les agents de planification et de génération utilisent gpt-5.1, l'agent de déploiement gpt-5.4-mini — plus léger pour une tâche déterministe à base de gabarits Jinja2 — et l'assistant RAG gpt-4o. Tous sur la même ressource Azure OpenAI EU. L'architecture est agnostique au modèle : c'est une variable d'environnement AZUREMODEL0x, ce qui m'a permis de faire évoluer le modèle pendant le développement sans toucher au code des agents. »

.env → AZUREMODEL01=gpt-5.1, AZUREMODEL03=gpt-5.4-mini, AZUREMODEL0RAG=gpt-4o.

Ne jamais dire « je ne sais plus quel modèle ». Le rapport corrigé est désormais aligné.

Q : Pourquoi pas Claude ou Gemini ?

« Critère bloquant : disponibilité dans le catalogue Azure OpenAI EU pour la résidence RGPD. Claude et Gemini n'y sont pas (endpoints hors Azure). »

Q : Pourquoi gpt-5.4-mini pour l'Agent 03 et pas gpt-5.1 ?

« Le déploiement génère des scripts via des gabarits Jinja2 déterministes : peu de raisonnement requis. Un modèle plus léger suffit, il est plus rapide et moins coûteux, à qualité équivalente sur cette tâche. »

B. SAW-AHP (votre point fort — maîtriser le calcul)

Q : Le SAW-AHP, c'est de l'IA ?

« Non, et la distinction est importante. SAW-AHP est une méthode de décision multicritère (MCDM) déterministe et explicable. L'IA dans mon système, c'est le LLM qui propose les services candidats et génère le Terraform. Le SAW-AHP arbitre entre les candidats de façon traçable. C'est le mariage IA (créatif) + MCDM (décisionnel auditable) qui rend les décisions défendables. »

Q : Comment dérivez-vous les poids ? Donnez le CR.

« Matrice de comparaison par paires sur l'échelle de Saaty, poids = moyenne géométrique normalisée par ligne. $\lambda_{\max} = 5,0783$, $CR = (5,0783 - 5) / (4 \times 1,12) = 0,0783 / 4,48 = 0,0175$, bien inférieur au seuil de 0,10. C'est calculé à l'exécution dans `ahp_weights.py` — je peux le lancer devant vous. »

`python core/ahp_weights.py` affiche le rapport complet.

Ne dites PAS $\lambda_{\max}=5,099$ (ancienne erreur corrigée). Le bon chiffre est 5,0783.

Q : Pourquoi inverser la complexité (1-c) ?

« Pour favoriser les migrations simples : un effort de migration faible doit augmenter le score, pas le diminuer. »

Q : SAW vs TOPSIS ?

« SAW est simple ($O(n \cdot m)$), interprétable, sans rank-reversal, et son CR est vérifiable. TOPSIS est plus sujet au rank-reversal. »

C. GRAPH RAG (distinguer du GraphRAG de Microsoft)

Q : Votre Graph RAG, c'est celui de Microsoft (Edge et al.) ?

« Je m'en inspire mais ne l'implémente pas. Le GraphRAG de Microsoft fait du résumé de communautés via LLM sur un graphe extrait de texte. Le mien combine une recherche vectorielle pgvector et la traversée d'un graphe de dépendances Terraform typées (DEPENDS_ON, COMPANION) construites à l'analyse statique. »

`rag/graph_rag.py`.

Q : Prouvez que le Graph RAG apporte quelque chose vs un LLM seul.

« Deux arguments. Empiriquement, un preprint (Nekrasov 2025) rapporte une progression du taux de succès global de 27,1 % à 62,6 % — je le cite comme indicatif, c'est non revu par les pairs. Mon argument structurel est plus fort :

les ressources Terraform ont des dépendances formelles typées qu'un RAG vectoriel rate ; mon graphe les reconstruit explicitement. »

Le « +131 % » est un taux de succès global, PAS une précision au sens IR. Les chiffres 83,2 %/62,6 % viennent de la Table 16 (variante GR-LLMSum) ; 37,2/70,2/80,3 de la Table 13. Ne mélangez pas les deux tables à l'oral.

Q : Pourquoi pgvector et pas Pinecone/Qdrant ?

« Extension dans la même instance PostgreSQL : zéro service tiers, transactions ACID partagées, résidence UE garantie. Pinecone/Qdrant sont externes → risque RGPD (Art. 44-49). »

Q : Quel index, IVFFlat ou HNSW ? Quelle dimension ?

« Embeddings all-mpnet-base-v2, 768 dimensions, exécutés localement (aucune donnée externe). » [Index : à confirmer dans rag/migrations ou alembic avant la soutenance — ne pas affirmer IVFFlat/HNSW sans vérifier.]

D. ARCHITECTURE & PIPELINE

Q : Pourquoi LangGraph et pas CrewAI/AutoGen ?

« Trois propriétés décisives absentes des autres : état persisté PostgreSQL (reprise après interruption), points d'arrêt natifs pour le Human-in-the-Loop, et boucles conditionnelles pour la validation. AutoGen n'a pas de reprise ; CrewAI a des transitions limitées. »

Q : Comment les agents communiquent-ils ?

« Jamais directement. Uniquement via l'état partagé MigrationState (TypedDict persisté). Chaque agent lit ses entrées, écrit ses sorties qui deviennent les entrées du suivant. C'est la figure 4.X (pipelines de données). Séparation des préoccupations stricte. »

runagent01/02/03(state: MigrationState) ; état dans agents/pipeline_state.py.

Q : Combien de nœuds, donnez un routeur conditionnel.

« 22 nœuds, 9 routeurs. Exemple : après l'analyse, un routeur teste si des fichiers laC ont été détectés → génération automatique sinon sélection manuelle des services. »

pipeline/pipelinegraph.py (22 addnode, 9 addconditionaledges).

Q : Combien de services Docker ? Neo4j est-il déployé ?

« 7 services, dont Neo4j. Neo4j accélère les traversées de graphe, mais le système a un fallback PostgreSQL WITH RECURSIVE : il tourne aussi en mode dégradé sans lui. »

Dire 7, pas 6. Le compose contient bien Neo4j.

Q : Prouvez la reprise après interruption.

« AsyncPostgresSaver de LangGraph persiste l'état à chaque nœud. Au redémarrage, le graphe reprend au dernier checkpoint via le threadid. Testé dans testpipeline_resume.py (37 cas, round-trip JSON). »

E. SÉCURITÉ

Q : Comment chiffrez-vous les secrets ?

« Deux niveaux : Fernet (AES-128-CBC + HMAC-SHA256) pour le chiffrement applicatif en base, et HashiCorp Vault pour la délivrance JIT au moment du terraform apply. Le token GitHub n'apparaît jamais en clair. »

core/tokenencryption.py, services/credentials/vaultstore.py.

Fernet = AES-128, pas AES-256 (corrigé dans la figure). Si on demande pourquoi pas AES brut : « un chiffrement par bloc sans authentification de message est vulnérable au bit-flipping ; Fernet intègre un HMAC. »

Q : JWT, quel algorithme ?

« HS256. Le serveur refuse de démarrer en production si JWTSECRETKEY fait moins de 32 caractères. »

api/auth/jwthandler.py (ALGO="HS256").

Q : Le code source part-il vers le LLM ?

« Non. L'analyse (Phase 0) est 100 % locale, par AST et HCL2. Vérifié par interception réseau httpx dans teststackanalyzernetwork.py (15 cas, 0 appel HTTP externe). »

Q : Rate limiting ?

« slowapi : 5 requêtes/minute sur l'authentification, jusqu'à 10/minute sur les autres endpoints sensibles, contre le brute-force. »

F. TESTS & VALIDATION

Q : Combien de tests, quelle couverture ?

« 661 cas collectés par pytest sur 31 fichiers — je peux le prouver avec pytest --collect-only. Couverture globale 35 %, mais 48 % sur core/. Le global est tiré vers le bas par les couches agents/ et pipeline/ qui dépendent d'Azure OpenAI et de Terraform réels, non reproductibles en unitaire isolé. J'ai donc testé en unitaire tout le déterministe et validé le non-déterministe par 3 scénarios end-to-end. »

Assumez le 35 % avec l'explication. Relancer pytest --cov la veille pour avoir le chiffre du jour.

Q : Une seule paire AWS→Azure testée E2E ?

« Oui, c'est une limite assumée. Le déploiement réel sur chaque cloud génère des coûts d'infrastructure incompatibles avec un PFE. L'architecture supporte les 6 paires (validateur d'état + template Jinja2 par provider), seule la validation E2E s'est limitée à AWS→Azure, opérationnelle chez Talan. »

G. CHOIX TECHNIQUES & LIMITES (assumer = gagner des points)

Q : Pourquoi Docker Compose et pas Kubernetes ?

« Docker Compose suffit pour le périmètre PFE — 7 services sur réseau interne. K8s serait la perspective de production pour scaler horizontalement l'executor. Choix de périmètre assumé, pas une lacune. »

Q : Le fast-path AIFASTPATH codé en dur ne contredit-il pas l'IA ?

« C'est un choix d'ingénierie. Trois services IA très verbeux troublent le scoring LLM générique ; pour ces cas précis j'ai un fast-path déterministe. Tout le reste passe par le scoring LLM + SAW-AHP. Principe : déterminisme quand on peut, IA quand on en a besoin. Dette technique identifiée, à remplacer par un enrichissement RAG. »

Q : Un choix que vous regrettez ?

« La couverture de tests sur les couches agents, et j'aurais clarifié plus tôt que Neo4j est optionnel. »

Q : Comment gérez-vous le tfstate en parallèle ?

« Chaque migration a son répertoire output/{migration_id}/ isolé → états séparés. La concurrence sur la file de jobs est gérée par SELECT FOR UPDATE SKIP LOCKED. Pour du multi-tenant production, je migrerais vers un backend tfstate distant verrouillé. »

H. POINTS À VÉRIFIER PERSONNELLEMENT AVANT LA SOUTENANCE

- [] Relancer pytest --cov → noter le vrai % de couverture du jour
- [] Index pgvector : IVFFlat ou HNSW ? (rag/migrations / alembic/versions)
- [] Température LLM et paramètres (agents/*/llm_config.py)
- [] Ouvrir .env et mémoriser les noms de modèles exacts
- [] Préparer une démo de secours déjà terminée (output/ contient des runs)
- [] Vérifier qu'aucune capture d'écran du chp5 n'affiche encore « GPT-4.1 »
- [] S'entraîner à lancer python core/ahp_weights.py (preuve du CR=0,0175)
- [] Confirmer le chiffre Infracost (27,21 USD) et Checkov (32 vérifs / 9 MEDIUM) dans output/